This listing defines the index for holding percolator queries. Note the following:

- It contains a `query` field to hold users' (failed) queries.
- The data type of the `query` field must be `percolator`.

The rest of the schema consists of the definition borrowed from the original `tech_books` index.

Just as we define fields with data types such as `text`, `long`, `double`, `keyword`, and so on, Elasticsearch provides a `percolator` type. The `query` field is defined as `percolator` in listing 12.29, and it expects a query as the field value, which we see shortly.

Now that our percolator index (`tech_books_percolator`) mapping is ready, the next step is to store queries. These queries usually don't return results to users (like the Python example).

In the real world, a user query that doesn't yield a result will be indexed into this `percolator` index. The process of collating users' failed queries into a `percolator` index can be done inside the search application, but unfortunately, discussing it is out of scope here. Just imagine that the query in listing 12.29 doesn't yield a result and is now sent to the `percolator` index to be stored. The next listing provides the code to store the query.

Listing 12.30 Storing the query to search for a book

```
PUT tech_books_percolator/_doc/1
{
  "query" : {
    "match":{
      "name":"Python"
    }
  }
}
```

The same query the user tried to search with but that failed to get a positive result

The listing shows the indexing of a query, which is unlike indexing a normal document. If you remember the document/indexing operations from earlier chapters, any time we index a document, we use a JSON-formatted document with name-value pairs. This code, however, has a `match` query.

The query in listing 12.30 is stored in the `tech_books_percolator` index with document ID 1. As you can imagine, this index keeps growing with failed searches. The JSON document consists of query(ies) issued by users that don't return positive results.

The final piece of the puzzle is to search the `tech_books_percolator` index when our stock is updated. As a bookshop owner, we are expected to stock up; the next time we receive new stock, let's assume the Python book is included. We can now index it into our `tech_books` index for users to search for and buy.

Listing 12.31 Stocking up (indexing) a Python book

```
PUT tech_books/_doc/4
{
  "name":"Python in Action",
```